NAME :- Lokare Mahir Abdul Majeed

ROLL NO :- 23AI34

BATCH :- 02

EXPERIMENT NO :- 02

AIM :-

Implement and compare SGD, Mini-batch GD, Momentum and Adam optimizers.

OBJECTIVE :-

1. To understand the concept of optimization in Deep Learning.
2. To implement different optimization techniques used for training neural networks.
3. To implement Stochastic Gradient Descent (SGD).
4. To implement Mini-batch Gradient Descent.
5. To implement Momentum-based optimization.
6. To implement Adam optimization.
7. To compare the convergence and loss of different optimizers.

THEORY :-

Optimization in Deep Learning :-

Optimization is the process of adjusting the weights and biases of a neural network to minimize the loss function. During training, the network calculates gradients using backpropagation, and the optimizer uses these gradients to update the model parameters.

The learning rate determines the amount by which the weights are adjusted during each update. A very high learning rate can cause unstable training, while a very low learning rate can make convergence slow.

1. Stochastic Gradient Descent (SGD) :-

SGD updates the weights using one training sample at a time. It produces faster individual updates but introduces randomness into the training process, which can result in a noisier loss curve. The basic update rule is: $W = W - \eta \nabla W L$, where η is the learning rate and $\nabla W L$ is the gradient of the loss with respect to the weights.

2. Mini-Batch Gradient Descent :-

Mini-Batch Gradient Descent divides the training dataset into small batches and updates the weights using each batch. It combines the stability of batch gradient descent with the faster updates of SGD. Typical batch sizes can be 32 or 64.

3. Momentum :-

Momentum improves gradient descent by considering previous weight updates along with the current gradient. It helps produce smoother updates and can improve convergence speed while reducing oscillations.

4. Adam :-

Adam is an optimization method that uses adaptive learning rates during training. It combines information from previous gradients to adjust the parameter updates. Adam can automatically adjust the learning rate during training.

Comparison :-

Optimizer	Update Method	Main Characteristic
SGD	One sample	Fast but noisy updates
Mini-Batch GD	Small batches	Balance between stability and speed
Momentum	Current + previous updates	Smoother and faster convergence
Adam	Adaptive updates	Automatically adjusts learning rate

The uploaded DL experiment also demonstrates comparing optimizer losses across training epochs and reporting the final loss for each optimizer.

PROCEDURE :-

1. Install the required Python libraries.
2. Import NumPy and Matplotlib.
3. Generate a suitable training dataset.
4. Define a neural network with hidden layers.
5. Define the forward propagation and loss calculation.
6. Calculate gradients using backpropagation.
7. Implement SGD optimization.
8. Implement Mini-Batch Gradient Descent.
9. Implement Momentum optimization.
10. Implement Adam optimization.
11. Train the neural network using each optimizer.
12. Record the loss after each epoch.
13. Plot the loss curves for all four optimizers.
14. Compare the final loss values and convergence behaviour.

CODE :-

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

X = np.linspace(-2, 2, 500).reshape(-1, 1)
y = 3 * X**2 + 2 * X + 1 + 0.2 * np.random.randn(500, 1)

class NeuralNetwork:
    def __init__(self):
        self.W1 = np.random.randn(1, 16) * 0.5
        self.b1 = np.zeros((1, 16))
        self.W2 = np.random.randn(16, 8) * 0.5
        self.b2 = np.zeros((1, 8))
        self.W3 = np.random.randn(8, 1) * 0.5
        self.b3 = np.zeros((1, 1))

    def forward(self, X):
        self.Z1 = X @ self.W1 + self.b1
        self.A1 = np.maximum(0, self.Z1)
        self.Z2 = self.A1 @ self.W2 + self.b2
        self.A2 = np.maximum(0, self.Z2)
        self.Z3 = self.A2 @ self.W3 + self.b3
        return self.Z3
```

```
    def backward(self, X, y, output):
        m = X.shape[0]
        dZ3 = (output - y) / m
        dW3 = self.A2.T @ dZ3
        db3 = np.sum(dZ3, axis=0, keepdims=True)
        dA2 = dZ3 @ self.W3.T
        dZ2 = dA2 * (self.Z2 > 0)
        dW2 = self.A1.T @ dZ2
        db2 = np.sum(dZ2, axis=0, keepdims=True)
        dA1 = dZ2 @ self.W2.T
        dZ1 = dA1 * (self.Z1 > 0)
        dW1 = X.T @ dZ1
        db1 = np.sum(dZ1, axis=0, keepdims=True)
        return dW1, db1, dW2, db2, dW3, db3

    def parameters(self):
        return [self.W1, self.b1, self.W2, self.b2, self.W3, self.b3]

def train(optimizer, epochs=150, batch_size=32, lr=0.01):
    model = NeuralNetwork()
    losses = []
    velocity = [np.zeros_like(p) for p in model.parameters()]
    m = [np.zeros_like(p) for p in model.parameters()]
    v = [np.zeros_like(p) for p in model.parameters()]
    beta1 = 0.9
    beta2 = 0.999
    epsilon = 1e-8
    t = 0

    for epoch in range(epochs):
        indices = np.random.permutation(len(X))
        if optimizer == "SGD":
            batches = [[i] for i in indices]
        else:
            batches = [indices[i:i + batch_size] for i in range(0, len(indices),
batch_size)]

        epoch_loss = 0
        for batch in batches:
            X_batch = X[batch]
            y_batch = y[batch]
            output = model.forward(X_batch)
            loss = np.mean((output - y_batch) ** 2)
            epoch_loss += loss
            gradients = model.backward(X_batch, y_batch, output)
            parameters = model.parameters()

            if optimizer == "SGD":
                for i in range(len(parameters)):
                    parameters[i] -= lr * gradients[i]
            elif optimizer == "Mini-Batch GD":
                for i in range(len(parameters)):
                    parameters[i] -= lr * gradients[i]
            elif optimizer == "Momentum":
                for i in range(len(parameters)):
                    velocity[i] = 0.9 * velocity[i] - lr * gradients[i]
                    parameters[i] += velocity[i]
            elif optimizer == "Adam":
                t += 1
                for i in range(len(parameters)):
                    m[i] = beta1 * m[i] + (1 - beta1) * gradients[i]
                    v[i] = beta2 * v[i] + (1 - beta2) * gradients[i] ** 2
                    m_hat = m[i] / (1 - beta1 ** t)
                    v_hat = v[i] / (1 - beta2 ** t)
```

```
                    parameters[i] -= lr * m_hat / (np.sqrt(v_hat) + epsilon)

        losses.append(epoch_loss / len(batches))
    return losses

optimizers = {
    "SGD": (0.001, 1),
    "Mini-Batch GD": (0.01, 32),
    "Momentum": (0.01, 32),
    "Adam": (0.01, 32)
}

results = {}
for name, (lr, batch_size) in optimizers.items():
    print(f"\nTraining with {name}")
    results[name] = train(name, epochs=150, batch_size=batch_size, lr=lr)
    print(f"Final Loss: {results[name][-1]:.6f}")

plt.figure(figsize=(10, 6))
for name, losses in results.items():
    plt.plot(losses, label=name)
plt.xlabel("Epochs")
plt.ylabel("MSE Loss")
plt.title("Optimizer Comparison")
plt.legend()
plt.grid(True)
plt.show()
```

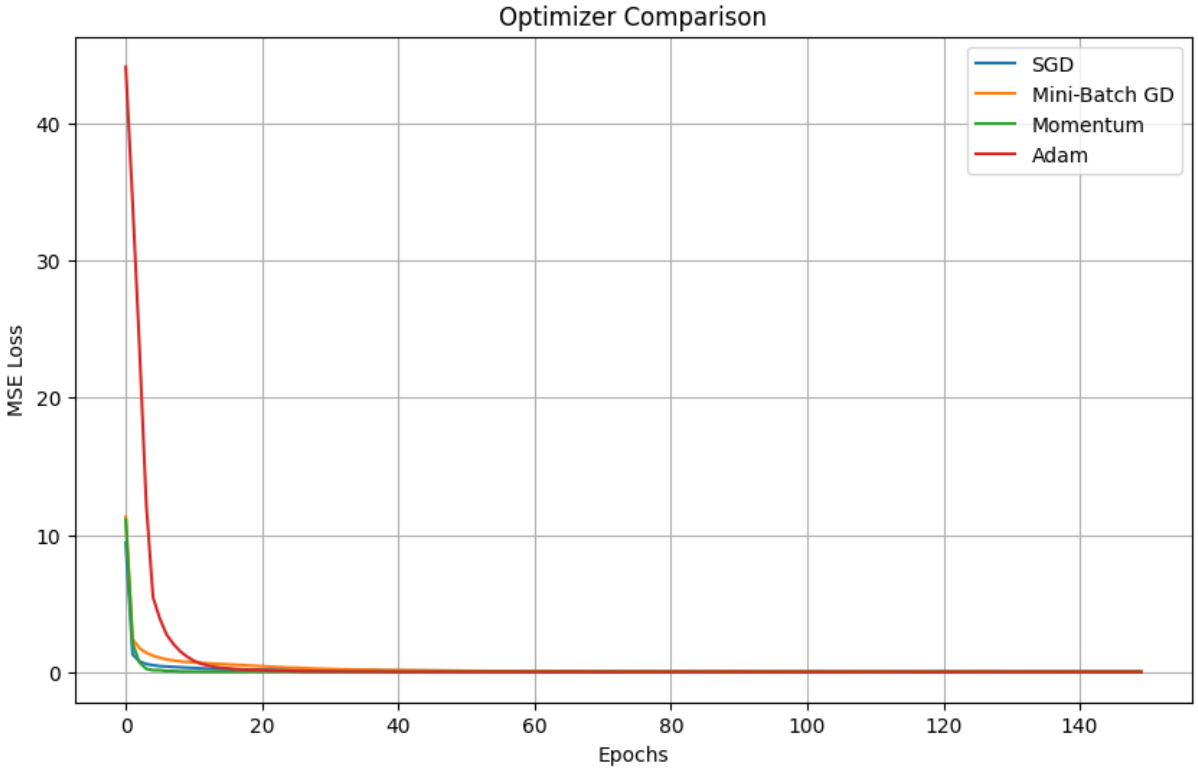
OUTPUT :-

```
Training with SGD
Final Loss: 0.047793

Training with Mini-Batch GD
Final Loss: 0.050054

Training with Momentum
Final Loss: 0.043071

Training with Adam
Final Loss: 0.046728
```



CONCLUSION :-

The experiment successfully implemented and compared SGD, Mini-Batch Gradient Descent, Momentum and Adam optimizers. The loss curves demonstrated the differences in their convergence behaviour, showing how the choice of optimizer and learning rate affects neural-network training. Momentum helps produce smoother updates, while Mini-Batch GD provides a balance between SGD and full-batch training.